

Cross-Platform Mobile App Development

Week 5 — React Native & Expo (Part 1): Core Architecture & Components

Instructor: Nguyen Thanh Tuan, PhD

Institution: Faculty of Computer Science, VKU

Today's Agenda

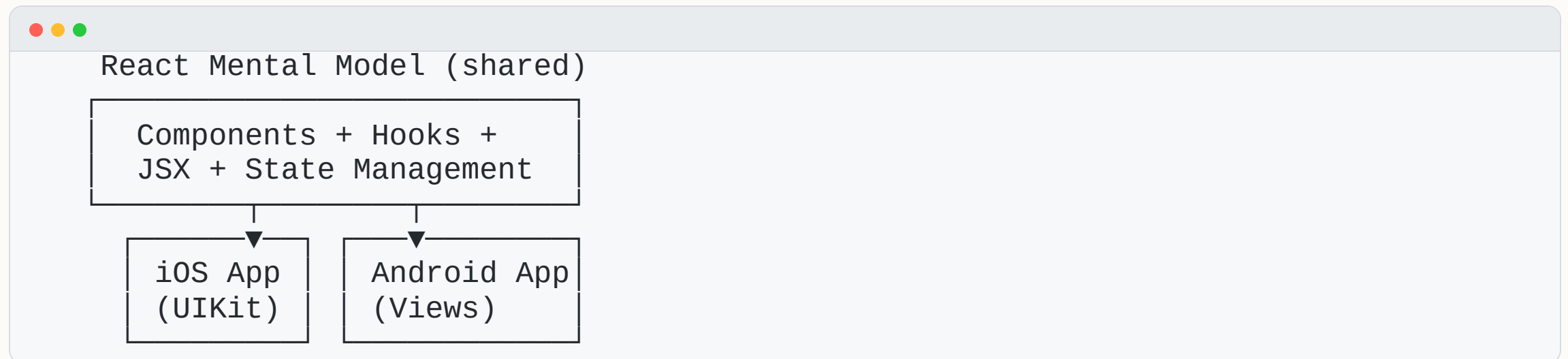
1. React Native Philosophy: "Learn Once, Write Anywhere"
2. Old Architecture (Bridge) vs. New Architecture (Fabric, TurboModules, JSI)
3. Expo Ecosystem: Managed Workflow vs. Bare Workflow
4. Core Native Components: `View` , `Text` , `Image` , `FlatList` , `TextInput` , `Pressable`
5. Styling in React Native: `StyleSheet` , Flexbox layout, and safe areas
6. Custom hooks for mobile UI and device dimensions
7. Mini-Project 2 Kickoff: Real-time Study Room Booking App

Part 1: Architecture Evolution

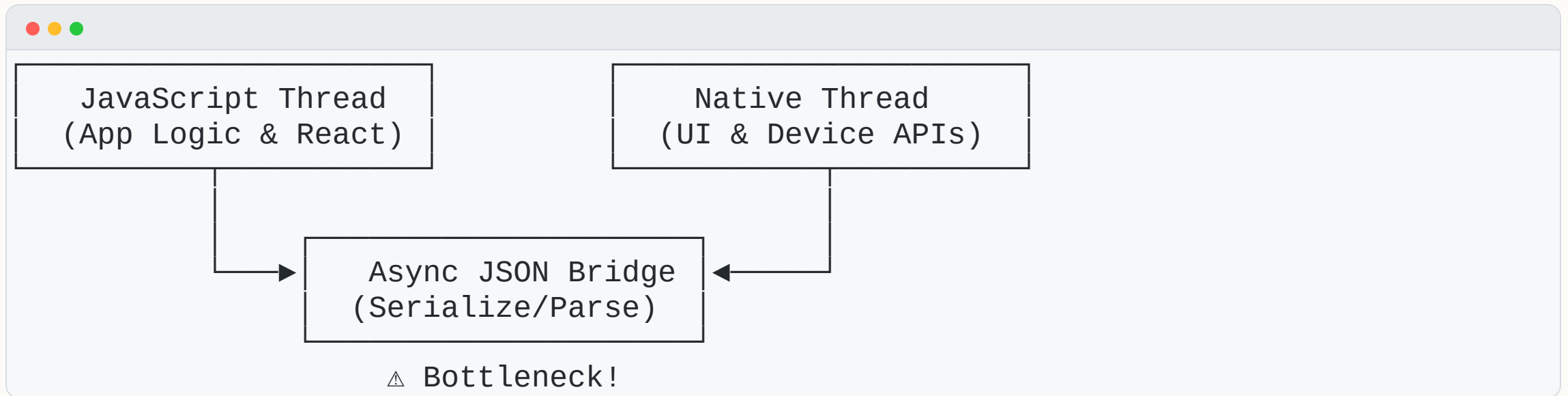
React Native Philosophy: "Learn Once, Write Anywhere"

Not "Write Once, Run Anywhere" (that's Java's broken promise). React Native acknowledges:

- iOS and Android have **fundamentally different** UX conventions (e.g. back gesture vs. hardware back button).
- You learn **one mental model** (React component tree + hooks) and apply it to **any platform**.
- Components render to **native platform widgets** — `<View>` becomes `UIView` on iOS, `ViewGroup` on Android.

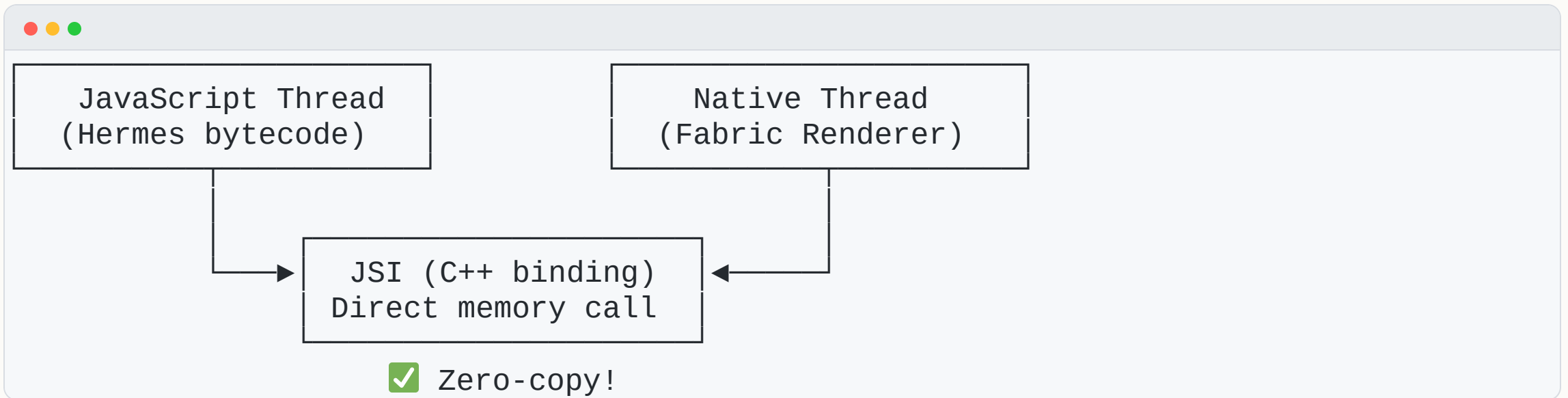


Old Bridge Architecture (Pre-2024)



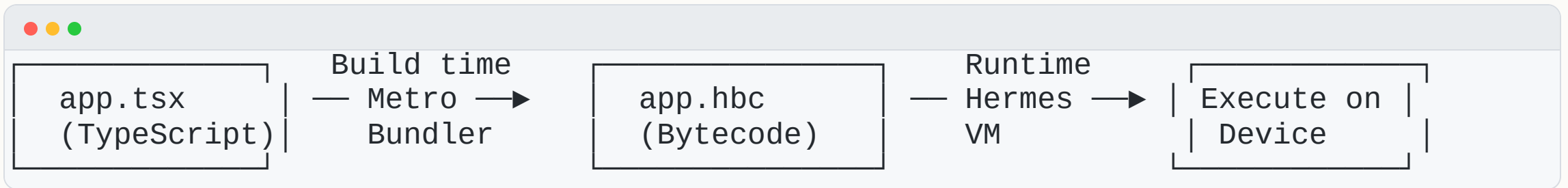
- Every JS ↔ Native call serialized as JSON string → parsed on the other side.
- **Async-only** — no synchronous layout measurements → visual "jumps" on first render.
- All native modules loaded at startup → slow cold boot on low-end devices.

The New Architecture (2026)



- **JSI:** Direct C++ memory invocation — no JSON bridge!
- **Fabric Renderer:** Synchronous UI layout for zero visual stutter.
- **TurboModules:** Lazy-loaded native modules (on-demand, not at startup).
- **Hermes:** Bytecode-compiled JS for sub-second cold boot.

Hermes Bytecode Compilation



Why Hermes matters for mobile:

- ⚡ **50% faster** cold start vs. JavaScriptCore (no parse step at runtime)
- 📉 **30% less** memory usage (bytecode is more compact than source text)
- 🛠 Built-in **Chrome DevTools** protocol for debugging over USB/WiFi

Part 2: Expo Ecosystem

Managed vs. Bare Workflow

Aspect	Managed Workflow	Bare Workflow
Setup	<code>npx create-expo-app</code> — ready in 30s	Eject or <code>npx react-native init</code>
Native code	✗ No <code>android/</code> or <code>ios/</code> folders	✓ Full access to native projects
Build	EAS Build (cloud)	Local <code>gradlew</code> / <code>xcodebuild</code>
OTA Updates	✓ <code>expo-updates</code> push JS bundles	✓ Same, but you manage signing
Custom native	Via Config Plugins	Direct Java/Kotlin or Swift/ObjC
Best for	90% of apps, rapid prototyping	Heavy native customization

Rule of thumb: Start with Managed. Eject only when you need a custom native module that has no Expo equivalent.

Creating an Expo Project

```
# Create a new project with the latest Expo SDK
npx create-expo-app@latest VKURoomBooking --template blank-typescript
```

Project structure

```
VKURoomBooking/
├── app.json           # App metadata & Expo config
├── App.tsx           # Root component
├── package.json
├── tsconfig.json
├── assets/           # Images, fonts, splash screen
└── node_modules/
```

```
# Start the dev server
```

```
npx expo start
```

```
# Press 'a' → Android emulator | 'i' → iOS simulator
```

```
# Scan QR code → open on Expo Go (physical device)
```

app.json — Key Configuration Fields

```
{
  "expo": {
    "name": "VKU Room Booking",
    "slug": "vku-room-booking",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#1E3A5F"
    },
    "ios": { "bundleIdentifier": "vn.edu.vku.roombooking" },
    "android": { "package": "vn.edu.vku.roombooking" },
    "plugins": ["expo-camera", "expo-location"]
  }
}
```

Part 3: Core Components & Layout

Core Components Reference

React Native Component	Native Android Equivalent	Native iOS Equivalent
<code><View></code>	<code>android.view.ViewGroup</code>	<code>UIView</code>
<code><Text></code>	<code>android.widget.TextView</code>	<code>UILabel</code>
<code><Image></code>	<code>android.widget.ImageView</code>	<code>UIImageView</code>
<code><FlatList></code>	<code>RecyclerView</code>	<code>UICollectionView</code>
<code><TextInput></code>	<code>android.widget.EditText</code>	<code>UITextField</code>
<code><Pressable></code>	Native Touch Listener	<code>UIControl</code> / Gesture Recognizer

⚠ No `<div>`, `<span>`, `<p>` tags! React Native compiles to **real native views**, not HTML.

<View> & <Text>: Component Code

```
import { View, Text, StyleSheet } from 'react-native';

function RoomCard({ name, building, capacity }: RoomCardProps) {
  return (
    <View style={styles.card}>
      <View style={styles.header}>
        <Text style={styles.roomName}>{name}</Text>
        <Text style={styles.badge}>{capacity} seats</Text>
      </View>
      <Text style={styles.location}>📍 {building}</Text>
    </View>
  );
}
```

<View> & <Text>: Styling

```
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff', borderRadius: 12, padding: 16,
    shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1, shadowRadius: 8, elevation: 3,
  },
  header: {
    flexDirection: 'row', justifyContent: 'space-between',
    alignItems: 'center',
  },
  roomName: { fontSize: 18, fontWeight: '700', color: '#1E293B' },
  badge: {
    fontSize: 12, color: '#fff', backgroundColor: '#3B82F6',
    paddingHorizontal: 8, paddingVertical: 4, borderRadius: 12,
    overflow: 'hidden',
  },
  location: { marginTop: 8, fontSize: 14, color: '#64748B' },
});
```

<Image>: Local vs. Remote

```
import { Image } from 'react-native';

// Local image (bundled at build time – fast, always available)
<Image source={require('./assets/room-placeholder.png')}
        style={{ width: '100%', height: 200, borderRadius: 12 }} />

// Remote image (MUST specify width & height)
<Image
  source={{ uri: 'https://api.vku.edu.vn/rooms/101/photo.jpg' }}
  style={{ width: '100%', height: 200, borderRadius: 12 }}
  resizeMode="cover"
/>
```

💡 **Tip:** For production, use `expo-image` for disk caching, blur hash placeholders, and animated transitions.

<FlatList>: High-Performance Lists

```
import { FlatList, View } from 'react-native';

function RoomListScreen({ rooms }: { rooms: Room[] }) {
  return (
    <FlatList
      data={rooms}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => (
        <RoomCard name={item.name} building={item.building}
          capacity={item.capacity} />
      )}
      ItemSeparatorComponent={() => <View style={{ height: 12 }} />}
      contentContainerStyle={{ padding: 16 }}
      initialNumToRender={10}
      maxToRenderPerBatch={5}
      windowSize={5}
    />
  );
}
```

<TextInput>: Controlled Input

```
import { TextInput, View, StyleSheet } from 'react-native';
import { useState } from 'react';

function SearchBar({ onSearch }: { onSearch: (q: string) => void }) {
  const [query, setQuery] = useState('');

  return (
    <TextInput
      style={styles.input}
      value={query}
      onChangeText={setQuery}
      placeholder="Search rooms..."
      placeholderTextColor="#94A3B8"
      keyboardType="default" // 'email-address' | 'numeric'
      returnKeyType="search"
      autoCapitalize="none"
      onSubmitEditing={() => onSearch(query)}
    />
  );
}
```

<Pressable>: Touch Feedback

```
import { Pressable, Text, StyleSheet } from 'react-native';

<Pressable
  style={({ pressed }) => [
    styles.btn,
    pressed && { opacity: 0.7 }    // Visual press feedback
  ]}
  onPress={() => onSearch(query)}
  onLongPress={() => clearSearch()}
  hitSlop={8}                    // Expand touch target by 8px
>
  <Text style={styles.btnText}>🔍 Search</Text>
</Pressable>
```

Why <Pressable> over <TouchableOpacity> ?

- **Pressable** is the modern replacement — supports **pressed** state, **hitSlop**, **android_ripple**.
- **TouchableOpacity** is legacy but still works.

Part 4: Styling & Flexbox Layout

StyleSheet.create() — Why Not Inline Objects?

```
// ✗ BAD: Creates new object on every re-render → GC pressure
<View style={{ flexDirection: 'row', padding: 16 }}>

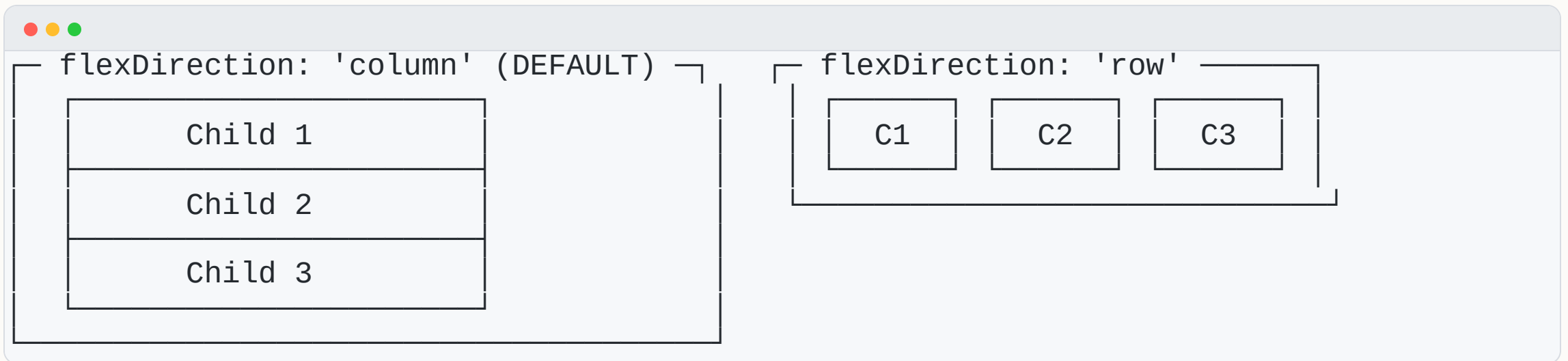
// ✓ GOOD: StyleSheet.create() freezes object, referenced by ID
<View style={styles.row}>

const styles = StyleSheet.create({
  row: { flexDirection: 'row', padding: 16 },
});
```

Benefits of `StyleSheet.create()` :

-  **Performance:** Styles sent to native once, referenced by numeric ID.
-  **Validation:** Catches typos at dev-time (`backgroundColor` not `background-color`).
-  **No CSS units:** Values are in **density-independent pixels (dp)**.

Flexbox Layout in React Native



⚠ **Key difference from CSS:** Default `flexDirection` is **column** (vertical), not **row**.

Flexbox Properties Reference

Property	Values	Description
<code>flexDirection</code>	<code>'column'</code> <code>'row'</code>	Main axis direction
<code>justifyContent</code>	<code>'flex-start'</code> <code>'center'</code> <code>'space-between'</code>	Align along main axis
<code>alignItems</code>	<code>'flex-start'</code> <code>'center'</code> <code>'stretch'</code>	Align along cross axis
<code>flex</code>	number	Proportion of remaining space
<code>gap</code>	number	Spacing between children

```
// Common pattern: horizontal row with space between
const styles = StyleSheet.create({
  row: { flexDirection: 'row', justifyContent: 'space-between',
    alignItems: 'center', padding: 16 },
  fill: { flex: 1 }, // Takes all remaining space
});
```

SafeArea: Handling Notches & Dynamic Island

```
import { SafeAreaProvider, SafeAreaView }  
  from 'react-native-safe-area-context';  
  
// Wrap once in App.tsx root  
export default function App() {  
  return (  
    <SafeAreaProvider>  
      <MainNavigator />  
    </SafeAreaProvider>  
  );  
}  
  
// Use in every screen  
function BrowseRoomsScreen() {  
  return (  
    <SafeAreaView style={{ flex: 1 }}  
      edges={['top', 'left', 'right']}>  
      <SearchBar onSearch={handleSearch} />  
      <RoomList rooms={rooms} />  
    </SafeAreaView>  
  );  
}
```

Part 5: Responsive Design & Custom Hooks

Custom Hook: `useResponsiveLayout`

```
import { useWindowDimensions, Platform, StatusBar } from 'react-native';

function useResponsiveLayout() {
  const { width, height } = useWindowDimensions();

  return {
    isLandscape: width > height,
    isTablet: width >= 768,
    columns: width >= 768 ? 3 : width >= 480 ? 2 : 1,
    cardWidth: width >= 768
      ? (width - 48 - 24) / 3
      : width - 32,
  };
}
```

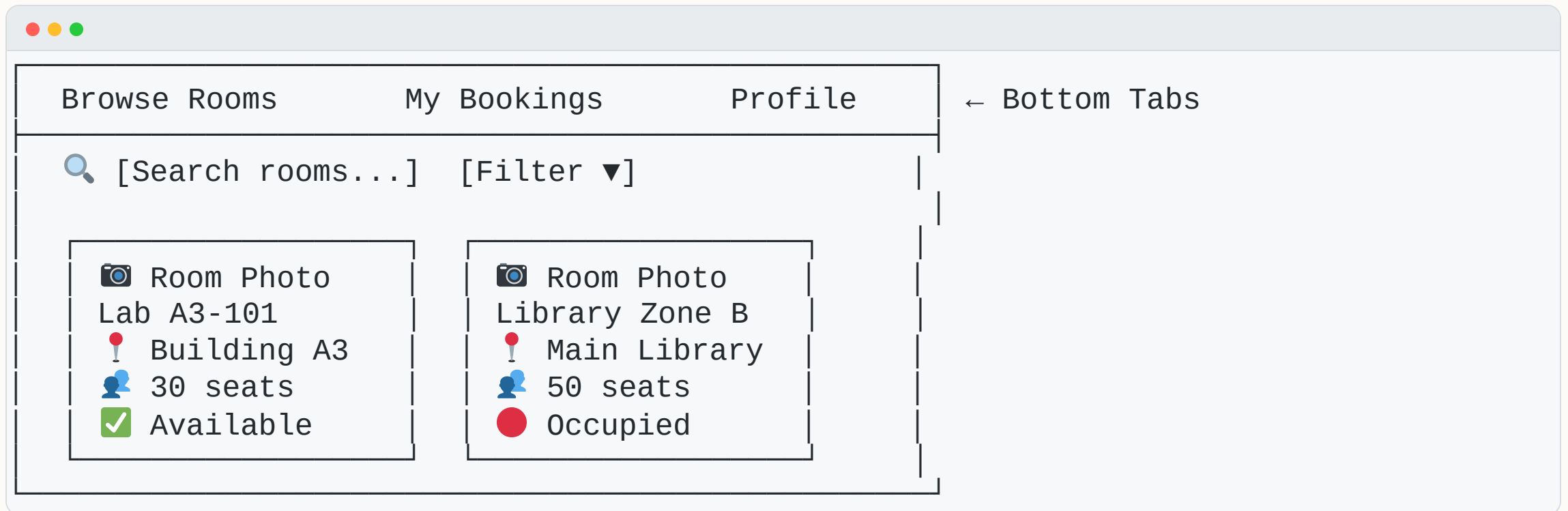
Using the Hook in a Component

```
function RoomGrid() {  
  const { columns, cardWidth } = useResponsiveLayout();  
  
  return (  
    <FlatList  
      data={rooms}  
      numColumns={columns}  
      key={columns} // Force remount when columns change  
      renderItem={({ item }) => (  
        <RoomCard style={{ width: cardWidth }} room={item} />  
      )}  
    />  
  );  
}
```

`useWindowDimensions()` automatically re-renders on rotation — unlike `Dimensions.get()` which is a one-time read.

Part 6: Mini-Project 2 Kickoff

Mini-Project 2: Study Room Booking App



Mini-Project 2 Requirements & Tech Stack

- **Goal:** Interactive mobile booking app for campus study rooms and labs.
- **Key Features:**
 - Room search & multi-parameter filter chips
 - 60fps `FlatList` feed with room cards
 - Time-slot selector with conflict prevention

Layer	Technology
Framework	React Native + Expo (Managed)
Language	TypeScript (strict mode)
Navigation	React Navigation 7 (Stack + Tabs)
State	Zustand (client) + TanStack Query (server)

- **Next Week:** Navigation and State Management with Zustand.

Today's Takeaways



The **New Architecture** (JSI + Fabric + TurboModules + Hermes) eliminates the async JSON bridge.



Expo Managed Workflow covers 90% of apps — start here, eject only when forced.



React Native components compile to **real native views**, not WebView HTML.



`StyleSheet.create()` + Flexbox (default `column`) is the styling model.



Always use `react-native-safe-area-context` for notches and Dynamic Island.

Before Week 6: Install Expo Go on your phone and experiment with core components.

Questions & Homework

1. Create a new Expo project and build a `RoomCard` component matching the wireframe.
2. Implement a `FlatList` with at least 20 mock room items.
3. Test on both a physical device (Expo Go) and an emulator.

Q&A